

# Metodología del Proyecto OpenMines

## 1. Panorama general

OpenMines es un entorno de simulación escrito en Python que modela el despacho de camiones en minas a cielo abierto. El proyecto persigue tres objetivos principales: (i) ofrecer un banco de pruebas flexible para comparar algoritmos de despacho clásicos y basados en IA; (ii) capturar la dinámica estocástica del sistema minero (averías, atascos, capacidad variable); (iii) proporcionar artefactos de análisis, visualización y soporte para aprendizaje por refuerzo. El motor está construido sobre `simpy`, organiza sus entidades en torno a la clase `Mine` y expone utilidades CLI, librerías y wrappers Gym para facilitar la experimentación.

## 2. Arquitectura de alto nivel

- **Núcleo de simulación** (`openmines/src/`): define entidades físicas (camiones, cargaderos, vertederos, carreteras) y lógica de orquestación.
- **Algoritmos de despacho** (`openmines/src/dispatch_algorithms/`): implementan estrategias concretas heredando de `BaseDispatcher`.
- **CLI y utilidades** (`openmines/src/cli/run.py`): generan escenarios, ejecutan experimentos, grafican y analizan logs.
- **Eventos y métricas** (`openmines/src/utlis/`): registran logs, transforman eventos en "ticks" y proveen herramientas de RL, visualización y análisis.
- **Configuraciones** (`openmines/src/conf/`): archivos JSON describen la topología de la mina y parámetros de simulación.

## 3. Pipeline de ejecución estándar

1. **Configuración**: el usuario selecciona un JSON (p.ej. `openmines/src/conf/north_pit_mine.json`) con definición de mina, flota, infraestructura y algoritmo(s) a evaluar.
2. **Construcción de objetos** (`run_dispatch_sim` en `openmines/src/cli/run.py`):
  - Se instancia `Mine` y se ata un `BaseDispatcher` concreto.
  - Se crean `ChargingSite`, `LoadSite`(s) con `Shovel`(s) y `DumpSite`(s) con `Dumper`(s).
  - Se configura la malla vial `Road` con matrices de distancia y parámetros de eventos.
  - Los camiones (`Truck`) se alojan en el `ChargingSite` listo para despachar.
3. **Arranque de la simulación** (`mine.start`):
  - Se generan procesos `simpy` por camión y monitores periódicos (mina, sitios, recursos, carreteras).
  - El tiempo avanza en el `Environment`; los camiones ceden el control cuando esperan recursos o se desplazan.
4. **Eventos y decisiones**:
  - Cada camión solicita al dispatcher órdenes `init`, `haul` y `back`; el envoltorio de `BaseDispatcher` actualiza colas y tiempos antes de cada decisión y registra métricas de latencia.
  - Se simulan averías, mantenimientos y atascos mediante `EventPool` y `RandomEventPool` asociados a camiones y carreteras.
5. **Finalización**:
  - Tras `sim_time` minutos simulados, `TickGenerator` produce un diccionario de ticks por minuto (posiciones, colas, producción, eventos).
  - Los resultados se guardan en `results/` y los logs en `logs/`; opcionalmente se generan gráficos (`Charter`) o animaciones (`VisualGrapher`).

## 4. Componentes núcleo

### 4.1 Mine (`openmines/src/mine.py`)

- **Rol**: nodo central que encapsula el estado global, el entorno `simpy`, la flota y los monitores.
- **Funciones clave**:
  - `add_*`: agrega sitios, carretera, dispatcher y setea referencias cruzadas.
  - `start` y `start_rl`: ejecutan la simulación clásica o asíncrona (cola de observaciones/acciones para RL).
  - `monitor_status`: recolecta indicadores longitudinales de producción, colas, eventos aleatorios y estados de camión; actualiza `self.status` por tick.
  - `update_road_status`: sintetiza congestiones, reparaciones y flujos de camiones en cada tramo para exponerlo a los algoritmos.
  - `dump_frames`: usa `TickGenerator` para materializar datos minuto a minuto (utilizado en CLI y visualización).
- **Interacciones**: manipula `EventPool` global, comparte `MineLogger`, consulta matrices de `Road` y delega la toma de decisiones al dispatcher asociado.

### 4.2 Truck (`openmines/src/truck.py`)

- **Rol**: modelo de recurso móvil; cada camión corre como proceso `simpy` independiente.
- **Ciclo de vida**:
  1. Sale del `ChargingSite` tras recibir `give_init_order`.
  2. Solicita recursos en la cola (`LoadRequest`) de un `Shovel`; durante la espera registra eventos y actualiza métricas de cola.
  3. Al cargar, acumula toneladas, incrementa contadores y evalúa mantenimiento de la pala.
  4. Pide `give_haul_order`, se desplaza a un `DumpSite` incorporando retrasos por atascos o averías.
  5. Pide `give_back_order`, descarga (`DumpRequest`) y retorna cargando estadísticas de ciclo.
- **Aleatoriedad**: `check_vehicle_availability` simula averías; `move` agrega demoras por eventos de carretera (`RoadEvent:jam`). Capacidad de quedar `unrepairable`.
- **Registro de eventos**: `EventPool` propio almacena cronograma de estados (espera, carga, viaje, etc.), usado por `TickGenerator` para reconstruir posiciones.

### 4.3 Sitios de carga y descarga (`openmines/src/load_site.py`, `openmines/src/dump_site.py`)

- **LoadSite**:
  - Agrupa `Shovel`(s), administra un `ParkingLot` para colas y controla indicadores: productividad, ratio de servicio, tiempos de espera.
  - Cada `Shovel` expone capacidad, ciclo, eventos de mantenimiento y un `simpy.Resource` (capacidad 1).
- **DumpSite**:

- Similar estructura con `Dumper(s)`; computa producción total, visitas, ratios de servicio.
  - Ambos ofrecen métodos `get_available_*` con heurística greedy (primer recurso libre o cola más corta).
- **ParkingLot**: monitoriza colas agregadas y por recurso utilizando los `Request` personalizados (heredados de `simpy.Resource`), actualizando tiempos estimados para alimentar al dispatcher.

#### 4.4 Road (`openmines/src/road.py`)

- Gestiona matrices de distancia: carga→descarga (`l2d_road_matrix`), descarga→carga (`d2l_road_matrix`) y carga desde `ChargingSite`.
- `check_availability` introduce reparaciones con tiempos aleatorios (exponencial + normal) y penaliza distancias.
- `road_jam_sampling` observa camiones en un tramo y, con distribuciones normales superpuestas, muestrea atascos (`RoadEvent:jam`) con duración Weibull.
- `truck_on_road` expone a cuántos camiones hay por segmento; `plot_road_jam_events` ayuda a depurar.

#### 4.5 BaseDispatcher (`openmines/src/dispatcher.py`)

- Clase abstracta que instrumenta `give_init_order`, `give_haul_order` y `give_back_order`.
- El decorador interno `_track_calls_and_time_wrapper` registra latencias, number de órdenes y secuencia de destinos, además de invocar `update_mine` para sincronizar colas y estados antes de cada decisión.
- Los algoritmos concretos (p.ej. `NearestDispatcher`, `RandomDispatcher`, `FixedGroupDispatcher`, `SPTFDispatcher`, `TabuSearchDispatcher`, etc.) se ubican en `openmines/src/dispatch_algorithms/`.

#### 4.6 Utilidades de soporte

- **MineLogger** (`openmines/src/utls/logger.py`): configura logging dual (consola + archivo) por simulación.
- **TickGenerator** (`openmines/src/utls/ticker.py`): sintetiza cada minuto datos detallados de camiones, sitios, palas, colas y eventos; alimenta visualización y análisis.
- **Event y EventPool** (`openmines/src/utls/event.py`): almacenamiento ordenado de eventos con búsquedas por tipo, descripción o intervalo de tiempo.

### 5. Sistema de eventos y estocasticidad

- **Eventos de camión**: averías (`TruckEvent:breakdown`), reparaciones, fallas irreparables. Gestionados por `Truck.event_pool` y replicados en `Mine.random_event_pool` para estadísticas globales.
- **Eventos de carretera**: congestiones y reparaciones producen penalizaciones de tiempo/distancia. Generados por `Road.road_jam_sampling` y `Road.check_availability`.
- **Mantenimiento de pala**: `Truck.load` evalúa mantenimientos estocásticos tras completar una carga.
- **Monitoreo**: `Mine.monitor_status` resume por tick KPI y contadores de eventos; `ParkingLot.update_queue_wait_status` recalcula tiempos esperados con la información de colas de `simpy`.

### 6. Entradas y configuraciones

Los archivos en `openmines/src/conf/` describen la mina. Componentes principales del JSON:

- `mine`: nombre del escenario.
- `dispatcher`: lista de algoritmos a ejecutar.
- `charging_site`: posición global y definiciones de flota (tipos, cantidades, capacidad, velocidad).
- `load_sites` y `dump_sites`: coordenadas, shovels/dumpers (con tiempos de ciclo y offsets), configuración de estacionamientos.
- `road`: matrices de distancia y parámetros de eventos (`lambda_repair`, medias/desviaciones para duración y penalizaciones).
- `sim_time`: duración total en minutos.

### 7. Ejecución vía CLI (`openmines/src/cli/run.py`)

- **Comando principal**: `openmines run -f <config.json>` o `openmines -f <config.json>`.
- **Funciones destacadas**:
  - `run_dispatch_sim`: ejecuta un solo dispatcher y devuelve ticks (estructura usada en tablas/gráficos y tests).
  - `run_simulation`: itera sobre los dispatchers declarados, acumula métricas y construye comparativas.
  - Visualización (`-v / visualize`): genera animación GIF a partir del archivo de ticks.
  - Ablaciones: `scene_based_fleet_ablation` y `algo_based_fleet_ablation` ajustan flota o escenarios y producen gráficas comparativas.
  - Análisis de logs: `openmines analyze <ruta>` invoca `LogAnalyzer` para resúmenes textuales.

### 8. Datos de salida y visualización

- Carpeta `results/`: contiene archivos JSON `MINE:{nombre}_ALGO:{algo}_TIME:{sim_time}.json` con ticks minuto a minuto.
- Carpeta `logs/`: bitácoras detalladas (`openmines_sim_<timestamp>.log`).
- `Charter` (`openmines/src/utls/visualization/charter.py`) genera curvas de producción, tablas y gráficos de ablation.
- `VisualGrapher` (`openmines/src/utls/visualization/graphher.py`) crea animaciones del estado de la mina.

### 9. Integración con Aprendizaje por Refuerzo

- **Wrapper**: `MineEnv` en `openmines/src/utls/r1_env.py` encapsula el flujo en un proceso separado para respetar `simpy` bajo Windows (`spawn`).
- **Comunicación**: colas `multiprocessing.Queue` transmiten observaciones y acciones entre el entorno y el agente.

- **Dispatcher para RL:** `RLDispatcher` ( `openmines/src/dispatch_algorithms/rl_dispatch.py` ) delega la acción a la política externa y mantiene acciones sugeridas ( `sug_dispatcher` ).
- **Wrapper Gym:** paquete `openmines/src/utils/gym/openmines_gym` registra entornos `mine/Mine-v1-dense|sparse` , exponiendo espacios de acción/observación compatibles con `Gymnasium`.
- **Uso típico:**

```
import gymnasium as gym
env = gym.make('mine/Mine-v1-dense', config_file='openmines/src/conf/north_pit_mine.json')
obs, info = env.reset()
for _ in range(1000):
    action = info['sug_action'] # heurística opcional
    obs, reward, done, truncated, info = env.step(action)
    if done or truncated:
        break
env.close()
```

## 10. Herramientas de optimización asistidas por IA

- `StrategyOptimizer` ( `openmines/src/optimizer.py` ) coordina iteraciones de generación/ejecución automática de despachadores apoyándose en modelos externos (`DeepSeek`, `Qwen`) vía API.
- **Flujo:** planeador → generador de código → ejecución `run_dispatch_sim` → análisis `LogAnalyzer` → retroalimentación.
- Mantiene historial ( `StrategyRecord` ), maneja reintentos ante errores de API y limpia módulos temporales.

## 11. Buenas prácticas para extender el sistema

- **Nuevos dispatchers:** crear un archivo en `openmines/src/dispatch_algorithms/` , heredar de `BaseDispatcher` , declarar atributo `name` y retornar índices válidos para cada orden. Aprovechar `mine.update_road_status()` y métricas de colas ya calculadas.
- **Nuevos eventos:** extender `EventPool` para registrar tipos adicionales; añadir lógica correspondiente en los monitores (p.ej. `Mine.monitor_status` ).
- **Escenarios:** duplicar un JSON en `openmines/src/conf/` , ajustar coordenadas, matrices y flota. Mantener coherencia dimensional entre `l2d_road_matrix` , `d2l_road_matrix` y cantidad de sitios.
- **Integraciones RL:** usar `MineEnv.make` para lanzar procesos y cerrar con `env.close()` para evitar huérfanos.
- **Pruebas:** módulos en `test/` (p.ej. `test/test_ppo.py` , `test/dqn.py` ) muestran cómo usar los wrappers RL y `CleanRL` para entrenamiento.

## 12. Referencias cruzadas rápidas

| Dominio                   | Archivo(s) clave                                                                                                                                                     | Responsabilidad                                |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| Núcleo de simulación      | <code>openmines/src/mine.py</code> , <code>openmines/src/truck.py</code>                                                                                             | Orquestación, procesos, eventos                |
| Infraestructura           | <code>openmines/src/load_site.py</code> , <code>openmines/src/dump_site.py</code> , <code>openmines/src/road.py</code> , <code>openmines/src/charging_site.py</code> | Modelos de sitios, colas, carreteras           |
| Despacho                  | <code>openmines/src/dispatcher.py</code> , <code>openmines/src/dispatch_algorithms/*.py</code>                                                                       | API y estrategias                              |
| Visualización y análisis  | <code>openmines/src/utils/ticker.py</code> , <code>openmines/src/utils/visualization/*</code> , <code>openmines/src/utils/analyzer.py</code>                         | KPIs, gráficos, reportes                       |
| RL                        | <code>openmines/src/utils/rl_env.py</code> , <code>openmines/src/utils/gym/openmines_gym</code>                                                                      | Wrapper Gym, interacción agente-entorno        |
| Optimización automatizada | <code>openmines/src/optimizer.py</code>                                                                                                                              | Loop de generación y evaluación de estrategias |

## 13. Conclusiones

OpenMines integra en un único repositorio un simulador granular de operaciones mineras, un ecosistema de algoritmos de despacho configurables, utilidades de análisis y soporte nativo a agentes de aprendizaje. La separación clara entre núcleo físico, estrategias y herramientas auxiliares permite extender el sistema con nuevos escenarios, eventos o políticas manteniendo coherencia interna. Para profundizar en cada componente, revisar los archivos listados y seguir el flujo descrito en esta metodología.